

GDsmith: Detecting Bugs in Cypher Graph Database Engines

Ziyue Hua, Wei Lin, Luyao Ren, Zongyang Li, Lu Zhang, Wenpin Jiao, Tao Xie

Peking University

ISSTA 2023

Background: Graph Databases and Cypher

Graph Databases (GDBs) store data based on the Property Graph Model:

- **Nodes** (entities like Developer, Project)
- **Relations** (connections like CONTRIBUTES, FOLLOWS)
- **Properties** on both (e.g., name= 'Michael ', stars=500)



Background: Graph Databases and Cypher

Graph Databases (GDBs) store data based on the Property Graph Model:

- **Nodes** (entities like Developer, Project)
- **Relations** (connections like CONTRIBUTES, FOLLOWS)
- **Properties** on both (e.g., name= 'Michael ', stars=500)



Query Language: Cypher

- A declarative, clause-based language: MATCH, WHERE, WITH, UNWIND, RETURN.
- Example:

```
MATCH (d:Developer)-[c:CONTRIBUTES]->(p:Project)
WHERE d.name = 'Michael' AND p.stars > 100
UNWIND p.languages AS lang
RETURN DISTINCT p.title, lang
```

Existing Testing Methods and Their Defects

- **Current State:** Several tools have been developed for database testing:
 - **RDBMS Tools:** **SQLsmith** (Random generation) and **SQLancer** (Logic-based testing) are widely used for SQL databases.
 - **GDB Tools:** **Grand** (Zheng et al., ISSTA '22) provides randomized differential testing for Gremlin-based engines.

Existing Testing Methods and Their Defects

- **Current State:** Several tools have been developed for database testing:
 - **RDBMS Tools:** **SQLsmith** (Random generation) and **SQLancer** (Logic-based testing) are widely used for SQL databases.
 - **GDB Tools:** **Grand** (Zheng et al., ISSTA '22) provides randomized differential testing for Gremlin-based engines.
- **Defects of Existing Methods:**
 - **Model Mismatch:** Relational database testing tools cannot fully exploit the unique characteristics of the **Property Graph Model**.
 - **Result Inefficiency:** Existing graph database testing tools lack techniques to increase the **ratio of queries with non-empty results**.

- **Core Idea:** Use the existing graph data to "guide" the generation of queries so that the query is guaranteed to match something.
- **Challenge 1: Pattern Matching Complexity**
 - *Challenge:* Complex patterns (e.g., long paths) rarely exist in random graphs.
 - *Solution:* **Graph-guided generation** - Extract patterns directly from subgraphs in the database.
- **Challenge 2: Unsatisfiable Conditions**
 - *Challenge:* Random WHERE clauses often conflict with data (e.g., `a.age > 20` when the node's age is 10).
 - *Solution:* **Data-guided generation** - Maintain a value table of current graph entities to build satisfiable expressions.

Algorithm 1 The top-level algorithm of GDsmith

Input: D_a : one instance of a graph database engine; D_b : the other instance of a graph database engine; N_g : the maximum number of newly generated queries.

Output: B : bug reports.

```
1: while the timeout does not exceed do
2:    $S \leftarrow \text{GENERATEPROPERTYGRAPHSCHEMA}(D_a, D_b)$ 
3:    $G \leftarrow \text{GENERATEPROPERTYGRAPH}(S, D_a, D_b)$ 
4:    $B \leftarrow \emptyset$ 
5:    $i \leftarrow 0$ 
6:   while  $i < N_g$  do
7:      $Q \leftarrow \text{GENERATECYPHERQUERY}(S, G)$ 
8:      $R_a \leftarrow \text{EXECUTECYPHERQUERY}(Q, D_a)$ 
9:      $R_b \leftarrow \text{EXECUTECYPHERQUERY}(Q, D_b)$ 
10:    if  $R_a \neq R_b$  or an exception is caught then
11:       $B \leftarrow B \cup \langle G, Q \rangle$ 
12:    end if
13:     $i \leftarrow i + 1$ 
14:  end while
15: end while
16: return  $B$ 
```

1 Graph Generation

Randomly defines a schema and populates a property graph.

2 Query Generation

- Generates a **Skeleton**.
- Completes it via **Graph-guided** and **Data-guided** strategies.

3 Differential Testing

Executes the query on multiple engines (Neo4j, Memgraph, etc.).

4 Bug Detection

Compares results; inconsistencies indicate potential bugs.

Stage 1: Schema and Property Graph Generation

- **Schema Definition:**

Randomly selects a set of labels $\{L_0, L_1, \dots\}$, relationship types $\{T_0, T_1, \dots\}$, and property keys $\{k_0, k_1, \dots\}$.

- **Graph Construction:**

- **Nodes:** Instantiates nodes, randomly assigning labels and key-value pairs (k_i, v_j) .
- **Relationships:** Connects node pairs with a type T_k and optional properties.

- **Metadata Storage:**

All generated entities and their concrete values are recorded to guide query generation in **Stage 2**.

Abstract Generation Example

1. Randomized Schema

Labels: $\{L_0, L_1\}$, Rel Types: $\{T_0\}$

Property Keys: $\{k_0, k_1\}$

2. Node Instantiation

$(n_0 : L_0\{k_0 : 10, k_1 : \text{'abc'}\})$

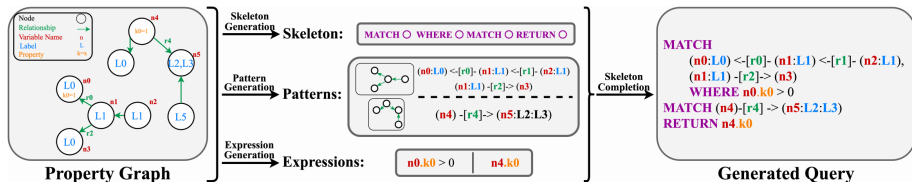
$(n_1 : L_1\{k_0 : 20\})$

3. Relationship Creation

$e_0 : (n_0) - [T_0\{k_1 : 5.5\}] \rightarrow (n_1)$

The Metadata records that a path exists from n_0 to n_1 where $n_0.k_0 = 10$.

Stage 2: Query Generation Overview



Following the workflow in **Figure 2**, query generation consists of:

- ❶ **Skeleton Generation:** Randomly selects a sequence of clauses (e.g., MATCH → WHERE → RETURN).
- ❷ **Pattern Filling:** Uses the graph data to instantiate MATCH patterns.
- ❸ **Expression Filling:** Fills WHERE or WITH expressions.
- ❹ **Semantic Analysis:** Ensures that all variable references and types are consistent across the query.

Stage 2: Graph-Guided Pattern Generation (Algorithm 2)

Goal: To generate complex Cypher MATCH patterns that are **guaranteed** to match at least one set of subgraphs in the database.

Key Variables:

- **Patterns:** The set of generated Cypher path patterns.
- **Vals:** A value table storing the ground-truth property values for each variable.
- **region:** A self-connected subgraph extracted from the graph G .
- **m :** The **Mapping Table** linking physical entities (N_i, R_j) to query variables (n_a, r_b).

Output: (Patterns, Vals) for subsequent clause generation.

Algorithm 2 The GENPATTERNS function

Input: G : the property graph.

Output: *Patterns*: the patterns that can be used to complete skeletons; *Vals*: the initial values of entity variables.

```
1: function GENPATTERNS( $G$ )
2:    $i \leftarrow 0$ 
3:    $Patterns \leftarrow \emptyset$ 
4:    $Vals \leftarrow \emptyset$ 
5:   while  $i < MaxRegionNumber$  do
6:      $region \leftarrow$  Randomly get one region (connected subgraph) from  $G$ 
7:      $m \leftarrow NewVars(region)$ 
8:      $Vals \leftarrow GETVALS(m) \cup Vals$ 
9:      $j \leftarrow 0$ 
10:    while  $j < MaxPatternInRegion$  do
11:       $g \leftarrow EXTRACTSIMPLESUBGRAPH(region)$ 
12:       $p \leftarrow TRANSLATEToPATTERN(g, m)$ 
13:       $Patterns \leftarrow Patterns \cup \{p\}$ 
14:       $j \leftarrow j + 1$ 
15:    end while
16:     $i \leftarrow i + 1$ 
17:  end while
18:  return  $Patterns, Vals$ 
19: end function
```

Workflow of Pattern Generation

- ➊ **Global Iteration:** Repeat the process to collect multiple path patterns (Patterns) and their corresponding ground-truth value table (Vals).
- ➋ **In Each Iteration:**
 - **Region Selection:** Randomly extract a connected subgraph (*region*) from the generated property graph G .
 - **Variable Mapping (m):** Map the nodes and relationships in the *region* to unique Cypher variable indices.
 - **Value Table Update:** Update Vals with the concrete property values of the mapped entities from the metadata.
- ➌ **Sub-iteration (Pattern Construction):**
 - Select a simple subgraph g within the current *region*.
 - Translate g into a Cypher pattern p (nodes and relationships) using the mapping m .
 - Append the newly generated pattern p to the Patterns set.
- ➍ **Finalization:** Return the complete Patterns and Vals. Vals will be used later to guide the generation of WHERE or WITH clauses.

A Concrete Example of Algorithm 2

Scenario: Translating Physical Data to Cypher Pattern

1. Physical Data (Stage 1 Metadata):

A path exists: $N_{114514}(\text{Label} : L_0, k_0 : 25) \xrightarrow{R_0(\text{Type}:T_0)} N_{233}(\text{Label} : L_1)$

2. Mapping and Val Table (m & $Vals$):

- **Mapping m :** $\{N_{114514} \rightarrow n_0, R_0 \rightarrow r_0, N_{233} \rightarrow n_1\}$
- **Value Table $Vals$:** $\{n_0.k_0 = 25, n_0.\text{label} = L_0, r_0.\text{type} = T_0, \dots\}$

3. Pattern Generation (p):

- Simple Subgraph g : $\{N_{114514}, R_0, N_{233}\}$
- Translate via m : $(n_0) - [r_0] - > (n_1)$
- Add Constraints: $p = (n_0:L_0) - [r_0:T_0] -> (n_1:L_1)$

4. Resulting Patterns:

MATCH $(n_0:L_0) - [r_0:T_0] -> (n_1:L_1)$

*Note: Since $Vals$ knows $n_0.k_0 = 25$, Stage 2 can now safely generate **WHERE** $n_0.k_0 > 20$ without fear of an empty result.*

Stage 2: Data-Guided Condition Generation (Algorithm 3)

Goal: To generate **satisfiable** expressions for various clauses (e.g., WHERE, WITH, RETURN), ensuring that the query execution flow remains active (non-empty results).

Key Components:

- **Value Table (Vals):** A "Ground-Truth" map maintained across clauses. It records the concrete values of all active variables to guide generation.
- **Target Constraint ($c, type$):** The goal passed to the generator.
 - WHERE: $c = true$
 - WITH/RETURN: $c = nil$
- **Top-Down Recursion:** Building complex expression trees by splitting parent constraints into sub-constraints for children.

Algorithm 3 The GENEXPR function

Input: c : a constraint that the generated expression needs to satisfy; $Vals$: the value table of variables; $type$: the expected value type of the expression.

Output: e : the expression tree that satisfies c and has the type $type$.

```
1: function GENEXPR( $c, Vals, type$ )
2:    $e \leftarrow$  Get one expression skeleton that has  $type$ 
3:   if  $e$  requires identifiers then
4:      $e.consts \leftarrow$  GENCONSTS( $e, c$ )
5:      $e.vars \leftarrow$  GENVARREFS( $e, c, Vals$ )
6:      $e.funcs \leftarrow$  GENFUNCREFS( $e, c$ )
7:   end if
8:    $childPosCons \leftarrow$  SPLITCONSTRAINTS( $e, c$ )
9:   for each  $p, cc \in childPosCons$  do
10:     $e.child[p] \leftarrow$  GENEXPR( $cc, Vals, TYPE(e.child[p])$ )
11:  end for
12:  return  $e$ 
13: end function
```

Workflow of Data-Guided Expression Generation

❶ Target Initialization:

- Predetermine the **Expected Value** (c) and the **Value Type** ($type$) for the current clause.
- *Example:* For a WHERE clause, c is set to `true` and $type$ is `Boolean`.

❷ Skeleton Selection (e):

- Select an **Expression Skeleton** (e.g., `>`, `<`, `AND`) from a pool that matches the required $type$.

❸ Recursive Constraint Splitting:

- Decompose the skeleton e into sub-components.
- If e is a **composite operator** (e.g., `>`), the algorithm splits the parent constraint c into new sub-constraints for its children (left/right expressions).
- Recursively call the generation function for these sub-expressions.

❹ Termination and Leaf Filling:

- If the skeleton is an **Identifier Expression** (e.g., variables like `a.v0` or constants like `255`), the recursion terminates.
- **Filling:** Consult the **Value Table** (`Vals`) to fill in concrete variables or constants that satisfy the required constraints.

A Concrete Example of Algorithm 3

Goal: Generate WHERE $a_0 > 5000$

1. Context from Vals:

The table records: $\{n_0.k_1 = 3000, n_1.k_1 = 4000\}$.

2. Intermediate Maintenance (WITH):

For WITH $n_0.k_1 + n_1.k_1$ AS a_0 :

GDsmith calculates $3000 + 4000 = 7000$ and updates Vals: $\{a_0 = 7000\}$.

3. Top-Down Generation (WHERE):

- **Constraint** c : $value = true$.
- **Operator Selection**: Selects ">"
- **Constraint Splitting**:
 - Left child must be a variable v .
 - Right child must be a constant k where $v.value > k$.
- **Filling**: Since $a_0(7000) > 5000$, it picks a_0 and 5000.

Resulting Clause:

WHERE $a_0 > 5000$ (Guaranteed to be TRUE)

Evaluation Setup

- **Subjects:** Three popular open-source graph database engines
 - **Neo4j** (Community Edition 3.5–4.4)
 - **RedisGraph** (v2.8)
 - **Memgraph** (v2.4)
- **Implementation:** GDsmith prototype with 22K+ LOC Java, built on SQLancer.
- **Baselines:** Two GDsmith variants for ablation study
 - GDsmith_{pc} : disables both graph-guided and data-guided techniques.
 - GDsmith_c : disables only data-guided condition generation.
- **Metrics:**
 - N_{VDD} : number of version-distinct discrepancies ($\approx$ unique bugs).
 - N_D : total discrepancies detected.
 - Non-empty-result query ratio.

Experiment Groups (Table 2)

Experiment	Engine Instances
Cross-engine	Neo4j 4.4.12, RedisGraph 2.8.20, Memgraph 2.4.0
Cross-version	Neo4j 4.4.12, Neo4j 3.5.0

Table: Experiment groups of GDsmith

- **Cross-engine:** detect inconsistencies among different engines.
- **Cross-version:** detect regressions between old and new versions of Neo4j.

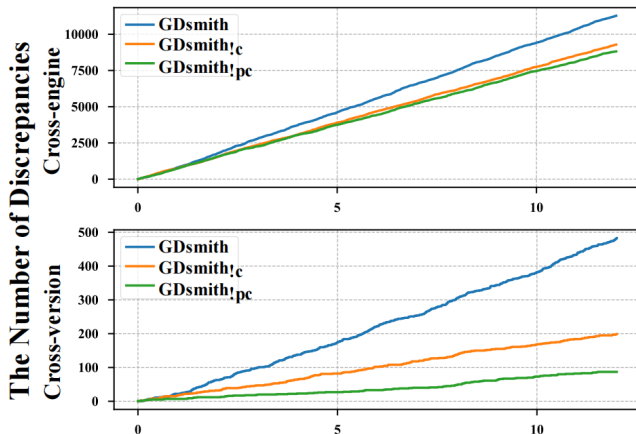
RQ1/RQ2: Effectiveness – Discrepancies Detected

Experiment	Approach	N_D	N_{VDD}
Cross-engine	GDsmith	11275	26
	GDsmith _c	9286	22
	GDsmith _{pc}	8815	17
Cross-version	GDsmith	483	18
	GDsmith _c	199	9
	GDsmith _{pc}	87	7

Table: Number of discrepancies (N_D) and version-distinct discrepancies (N_{VDD}) detected in 12 hours.

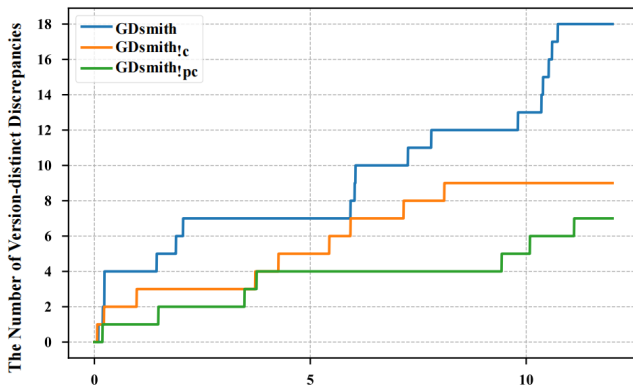
- GDsmith outperforms both baselines in all metrics.
- Both techniques contribute to effectiveness.
- Cross-engine testing generates significantly more discrepancies due to implementation differences.

RQ1/RQ2: Discrepancies Over Time (Figure 4)



- GDsmith's N_D grows fastest in both experiments.
- Data-guided technique (GDsmith_c) contributes more than pure random (GDsmith_{pc}).

RQ1/RQ2: Version-Distinct Discrepancies Over Time (Figure 5)



- GDsmith detects more unique bugs faster.
- Confirms that both techniques help reveal diverse bugs.

RQ1/RQ2: Non-Empty-Result Query Ratio (Table 4)

Approach	Non-Empty-Result Query Ratio
GDsmith _{pc}	18.19%
GDsmith _c	36.79%
GDsmith	73.66%

Table: Non-empty-result query ratio after 1 hour of generation.

- Graph-guided and data-guided techniques **jointly** increase the ratio.
- Higher ratio $\Rightarrow$ more opportunities to detect wrong-result bugs.

RQ3: Real-World Bugs Detected (Table 5)

Subject	Bug _{cr}	Bug _{wr}	Fixed	Confirmed	Reported
Neo4j	5	2	7	0	0
RedisGraph	3	11	6	7	1
Memgraph	0	7	1	2	4
Total	8	20	14	9	5

Table: Bug detection results of GDsmith (Bug_{cr}: crash bugs, Bug_{wr}: wrong-result bugs).

- 28 bugs total, 23 confirmed, 14 already fixed.
- Demonstrates practical value of GDsmith.

Example Bug 1: Wrong Result in RedisGraph

Trigger Query (Simplified)

```
CREATE (n0:T2), (n1)<-[r1:T1{id:2,k51:1}]->(n2);  
MATCH (n7) OPTIONAL MATCH (n0:T2), (n7)<-[r7:T1]->(n8)  
RETURN (r7.k51) AS a0;
```

- **Expected:** a0 = 1, null, null
- **RedisGraph output:** a0 = 1, 1, 1
- **Root cause:** Query operator Argument misplaced in the operator tree.
- **Why GDsmith found it:** Non-empty result + complex pattern combination exposed the wrong planning.

Example Bug 2: Wrong Result in Neo4j

Trigger Query (Simplified)

```
CREATE (n0), (n1);  
MATCH (n0) OPTIONAL MATCH (n3) WITH n3  
OPTIONAL MATCH (n3), (n5) OPTIONAL MATCH (n3) RETURN 1;
```

- **Expected:** One row (value 1)
- **Neo4j 4.2.14 output:** Two rows (value 1, 1) — rows doubled.
- **Root cause:** Planner mistakenly used AllNodesScan for pattern n3, multiplying rows.
- **Key:** Combination of multiple OPTIONAL MATCH and WITH clauses triggered the bug.

Summary of Contributions

- **First approach** for automated testing of Cypher graph database engines.
- **Two novel techniques:**
 - Graph-guided pattern generation
 - Data-guided condition generation
- **Extensive evaluation:**
 - Outperforms baselines in discrepancy detection and non-empty query ratio.
 - Detected **28 real bugs** in Neo4j, RedisGraph, and Memgraph.
- **Open-source implementation** available at <https://github.com/ddaa2000/GDsmith>.